# Improving Dialogue State Tracking through Combinatorial Search for In-Context Examples

**Haesung Pyun**[1] **Yoonah Park**[2] **Yohan Jo**[1]*
[1]Graduate School of Data Science, Seoul National University
[2]Department of Computer Science and Engineering, Seoul National University
{haesung.pyun, wisdomsword21, yohan.jo}@snu.ac.kr

## Abstract

In dialogue state tracking (DST), in-context learning comprises a retriever that selects labeled dialogues as in-context examples and a DST model that uses these examples to infer the dialogue state of the query dialogue. Existing methods for constructing training data for retrievers suffer from three key limitations: (1) the synergistic effect of examples is not considered, (2) the linguistic characteristics of the query are not sufficiently factored in, and (3) scoring is not directly optimized for DST performance. Consequently, the retriever can fail to retrieve examples that would substantially improve DST performance. To address these issues, we present CombiSearcha method that scores effective in-context examples based on their combinatorial impact on DST performance. Our evaluation on MultiWOZ shows that retrievers trained with CombiSearch surpass state-of-the-art models, achieving a 20Œ gain in data efficiency and generalizing well to the SGD dataset. Moreover, CombiSearch attains a 12% absolute improvement in the upper bound DST performance over traditional approaches when no retrieval errors are assumed. This significantly increases the headroom for practical DST performance while demonstrating that existing methods rely on suboptimal data for retriever training.[1]

## 1 Introduction

Dialogue state tracking (DST), a core task for task-oriented dialogue systems and tool agents, tracks user intentions and relevant constraints throughout the dialogue. For example, if a user says "I want to book a guest house" and later adds "I want to book it for four people", the dialogue state is updated with the slot-value pairs '[<hotel-type: guest house>, <hotel-book people: 4>]'.

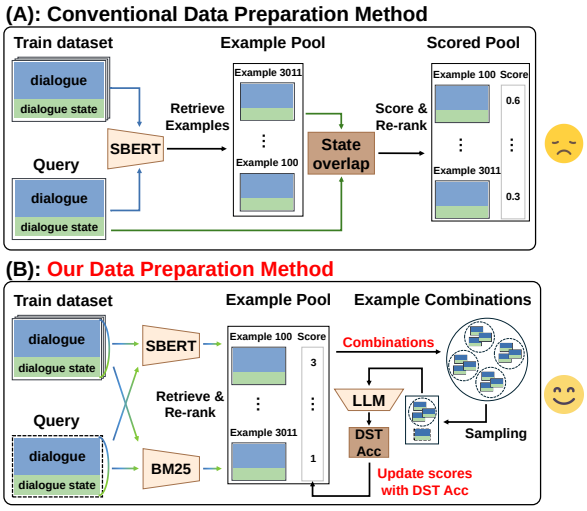



Figure 1: (A) Conventional data preparation method. (B) Our data preparation method that integrates diverse pool construction and combinatorial re-ranking based on DST performance.

Recent work shows that large language models can perform DST through in-context learning (ICL) (Brown et al., 2020; Zhao et al., 2023). By supplying queryresponse exemplars in the prompt, models learn task-specific patterns from in-context examples, thereby eliminating the need for costly fine-tuning or extensive annotation (Brown et al., 2020; Zhang et al., 2022; Chowdhery et al., 2023; Hoffmann et al., 2022; Touvron et al., 2023). However, since performance depends on the relevance of these exemplars (Liu et al., 2022), recent studies propose retrievers that dynamically select examples tailored to each test query, boosting the accuracy and robustness of ICL performance across tasks (Levy et al., 2023; Lu

*Corresponding author
[1]Our code: https://github.com/holi-lab/combisearch

et al., 2023; Ye et al., 2023).

To select effective examples, it is crucial to train an ICL example retriever on high-quality data. The most widely adopted approach for preparing retriever training data follows two steps (Hu et al., 2022): (1) **pool construction** selects dialogues similar to the target query, and (2) **re-ranking** prioritizes dialogues whose dialogue states substantially overlap with the querys dialogue state (Figure 1-A). However, what constitutes effective in-context examples for DST and how to systematically select them remain understudied and particularly challenging, as DST requires a comprehensive understanding of various conversational aspects (Chen et al., 2023). For instance, coreference resolution is both crucial and error-prone, requiring the system to correctly identify anaphors (e.g., "it") and their corresponding antecedents (e.g., "guest house"), which may span multiple dialogue turns. Our analysis identifies prominent errors and challenges in prior approaches (Appendix A.1).

We focus on three key limitations in the prior data preparation process and address them in this paper. First, the re-ranking step scores each example individually, which can result in selecting redundant in-context examples that offer little new information. Second, relying solely on dialogue state similarity may neglect important contextual information within the dialogue itself and retrieve examples whose conversational characteristics differ significantly from the query dialogue. Such examples are suboptimal for handling complex linguistic phenomena, such as coreference resolution, present in the query dialogue. Furthermore, re-ranking examples based on dialogue state similarity does not directly optimize for DST performance, resulting in only "indirect supervision".

To address these limitations, we introduce CombiSearch, a new method for preparing training data for in-context example retrievers in DST. By combining diverse pool construction with combinatorial re-ranking (Figure 1-B), CombiSearch has three key characteristics. First, the pool construction step collects diverse dialogues that share similar linguistic characteristics with the query—such as conversational flow, style, and structure—beyond just dialogue states. This ensures that more relevant examples are considered during re-ranking, improving the system's ability to handle the linguistic phenomena present in the query. Second, rather than re-ranking individual examples in isolation, CombiSearch scores examples based on how they function together in tandem. By evaluating their combinatorial impact on DST performance, it accounts for synergistic effects that might be overlooked when scoring examples separately. Finally, the re-ranking step scores examples by directly optimizing Joint Goal Accuracy (JGA), a standard metric for DST, rather than relying on the indirect signal of dialogue state similarity between each example and the query.

According to our evaluation, CombiSearch outperforms existing state-of-the-art methods on both the MultiWOZ 2.4 and Schema-Guided Dialogue (SGD) datasets. Notably, CombiSearch is highly data-efficient, enabling a retriever trained on CombiSearch-scored data to surpass all baseline models while using only 5% of the training data required by these models. Understanding the upper bound performance achievable with retriever training data *without retriever errors* is crucial, as this sets a hard limit on what practical retrievers can achieve. Under this ideal assumption of no retrieval errors, using examples scored by CombiSearch outperformed earlier approaches by 12% JGA, highlighting that prior methods rely on suboptimal data for training retrievers and that a more effective data preparation approach like CombiSearch is imperative for breakthroughs in DST. Notably, all these advances from CombiSearch's combinatorial nature come with a running time that is linear in the number of examples to score.

Our key contributions are as follows:

1. We present CombiSearch, an effective combinatorial method for constructing data to train retrievers of in-context examples in DST.
2. We demonstrate that CombiSearch outperforms baselines in DST, even with 20x data efficiency.
3. We reveal the significantly improved upper bound performance of CombiSearch when no retrieval errors are assumed, highlighting the importance of optimizing retriever

training data.

## 2 Problem Setting

**Dialogue State Tracking** DST focuses on predicting the dialogue state that captures the users intentions and goals, based on both the current user-agent interaction and the conversation history. Let $A^t$ and $U^t$ denote the agents and users utterances at turn $t$, respectively. The dialogue history up to turn $t$ is $C^t = [(A^1, U^1), (A^2, U^2), \ldots, (A^t, U^t)]$. The objective is to infer a dialogue state $y^t$ from $C^t$, where $y^t$ is a set of slot-value pairs: $y^t = \{(s^1, v^1), (s^2, v^2), \ldots, (s^n, v^n)\}$.

Following Hu et al. (2022), we adopt a state-change formulation. Specifically, the model takes the previous dialogue state $y^{t-1}$ and current utterances $(A^t, U^t)$ as input, denoted as $x^t = [y^{t-1}, (A^t, U^t)]$, where $y^{t-1}$ summarizes $C^{t-1}$. It predicts the dialogue state change $\Delta y^t = \{+(s^i, v^i), \ldots, -(s^j, v^j)\}$, with "+" indicating addition and "" indicating removal of slot-value pairs. The $t$-th dialogue state is $y^t = y^{t-1} \cup \Delta y^t$.

**In-Context Learning for DST** Given a test query $x_{\text{test}}$, the ICL approach retrieves $k$ labeled examples $\{(x_i, \Delta y_i)\}_{i=1}^{k}$ from the training set $D_{\text{train}} = \{(x_i, \Delta y_i)\}_{i=1}^{N}$. The goal is to predict the dialogue state change of the test query $x_{test}$:

$$\Delta \hat{y}_{\text{test}} \sim P_{\text{LLM}}\big(y_{\text{test}} | \{(x_i, \Delta y_i)\}_{i=1}^{k}, x_{\text{test}}\big) \quad (1)$$

where $\sim$ refers to a decoding method (e.g., greedy decoding, beam search, etc.).

In conventional retriever-based ICL for DST, two stages are involved. *Example-Retrieval Training* trains the retriever on query-example pairs: each query is paired with candidates ranked by dialogue state similarity, and a contrastive loss draws similar embeddings together while repelling dissimilar ones. *Inference Phase* uses the trained retriever to fetch top-ranked examples based on the test query; these examples are prepended to the prompt so the language model can exploit them to generate the querys dialogue state.

## 3 Method

We present **CombiSearch**, an algorithm that aims to identify an optimal combination of in-context examples to improve overall DST performance. We first detail each step of CombiSearch (§3.1) and introduce a retriever training method based on CombiSearch-scored data (§3.2). Lastly, we describe our text-to-JSON prompt format (§3.3). The overall procedure is illustrated in Figure 2.

### 3.1 CombiSearch

CombiSearch is designed to generate training data for a retriever of in-context examples for DST. The resulting data consists of <query, scored examples> pairs which can be used to train a retriever to select high-scoring examples for a given query. CombiSearch unfolds in two complementary steps. First, for each query, it constructs a diverse pool of examples $E$ by leveraging both semantic and lexical similarity to the query. Second, it employs a combinatorial mechanism to score the examples based on their collective contribution to successfully predicting the query's dialogue state. The algorithm is presented in Algorithm 1. Please note that during data preparation and retriever training, each query is drawn from the training set.

**Pool Construction (Figure 2-1)** Our motivating observation is that DST errors often arise when essential linguistic phenomena required for accurately predicting the query's dialogue state (e.g., coreference resolution) are not adequately represented in the in-context examples. Hence, it is important to include diverse examples covering a broader range of linguistic phenomena relevant to the query. To that end, we adopt a hybrid search that integrates both semantic and lexical aspects of similarity using two retrievers: SBERT (Reimers and Gurevych, 2019) and BM25 (Robertson and Zaragoza, 2009). SBERT captures the semantic meaning of dialogue states while BM25 effectively captures linguistic properties—such as word choices, the use of coreferences, named entities, domains, and slots (Appendix D.2). We first retrieve the top-$N$ examples from each retriever and merge them into a single list. We then re-rank them based on a hybrid score defined as the product of BM25 TF-IDF score and SBERT cosine similarity score (Appendix E.3). Finally, we retain the top-$N$ examples to form the final pool $E$.

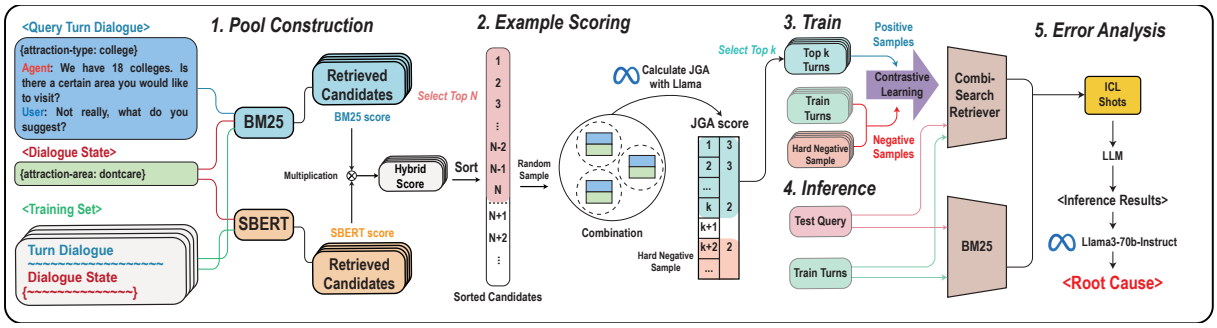



Figure 2: Overall structure of CombiSearch.

**Algorithm 1** Pseudo-code for **CombiSearch**

**Input:** $D_{\text{train}}$: Training Dataset, $D_{\text{query}}$: Query Dataset
**Output:** $D_{\text{CombiSearch}} = \{(x_i, y_i, E_i)\}_{i=1}^{|Q|}$
**Parameters:** $k$: Number of in-context examples, $M$: Number of evaluations per example, $P_i$: Input prompt

**for** $(x_{\text{test}}, y_{\text{test}}) \in D_{\text{query}}$ **do** ▷ Pool Construction
  $D_i \leftarrow \text{top-k}_{\text{BM25}}(D_{\text{train}}, N)$
  $D_j \leftarrow \text{top-k}_{\text{SBERT}}(D_{\text{train}}, N)$
  $D_{\text{union}} \leftarrow D_i \cup D_j$
  **for** $u \in D_{\text{union}}$ **do**
    $u.\text{hybrid_score} \leftarrow u.\text{TF-IDF} \times u.\text{cos_sim}$
  **end for**
  $E \leftarrow \text{top-k}_{\text{hybrid_score}}(D_{\text{union}}, N)$
  **for** $i = 1$ **to** $M$ **do** ▷ Example Scoring
    $\text{R}_\text{i} \leftarrow \text{select_random_subset}(E, k)$
    $\text{jga}_i \leftarrow \text{JGA}(\text{LLM}(P_i, x_{\text{test}}), y_{\text{test}})$
    **for** $e \in P_i$ **do**
      $e.\mathit{CombiScore} \leftarrow e.\mathit{CombiScore} + \text{jga}_i$
    **end for**
  **end for**
  $D_{\text{CombiSearch}} \leftarrow D_{\text{CombiSearch}} \cup \{(x_{\text{test}}, y_{\text{test}}, E)\}$
**end for**
**return** $D_{\text{CombiSearch}}$

**Example Scoring (Figure 2-2)** After constructing the candidate pool $E$ with $N$ examples per query, we score individual examples. Rather than scoring an example solely based on its similarity to the query as in existing work, we directly measure its impact on DST performance when used in ICL in tandem with other examples. This approach provides more direct supervision. Our objective is to identify an optimal set of examples that collectively enhance DST performance.

However, exhaustively evaluating all possible combinations is impractical due to the combinatorial explosion. Instead, we conceptualize each example as a key "team player" if it consistently contributes to the success of the "teams" it belongs to (i.e., accurate DST) across different sampled combinations. CombiSearch implements this by iteratively sampling $k$ examples and evaluating the DST model's prediction using JGA. A JGA score of 1 increases the score of every example in the sampled combination by 1, while a score of 0 leaves the scores unchanged. Repeating this sampling process $M$ times yields an accumulated score for each example—termed **CombiScore**—reflecting its contribution to DST. Notably, this algorithm allows CombiSearch to run in linear time with respect to the number of examples to score while preserving its combinatorial nature. The resulting dataset $D_{\text{CombiSearch}}$ pairs each query with a pool of examples $E$ ranked by their CombiScores.

### 3.2 Retriever Training (Figure 2-3)

We train a specialized retriever to identify effective in-context examples for each query. The retriever takes a dialogue ($y^{t-1}$, $A^t$, and $U^t$) as input and produces an embedding. The training objective is to maximize the embedding similarity between a query and its high-scoring examples in $D_{\text{CombiSearch}}$.

We construct training instances as follows. For each query $x_i$, we first construct a set of positive examples consisting of the top $|P|$ examples ranked by CombiScore from pool $E$; ties are broken by the degree of dialogue state overlap. Next, for every pair of $x_i$ and a positive example $e_j^+$, a set of negative examples is constructed by selecting the bottom $B$ examples from $E$ and additional $B-1$ examples randomly sampled from outside $E$. As a result, the training data comprise tuples of the form $\langle x_i, e_j^+, e_{j,1}^-, \ldots, e_{j,2B-1}^- \rangle$ for all $j = 1, ..., |P|$.

Based on this data, we fine-tune our retriever using the InfoNCE loss (van den Oord

et al., 2019):

$$L(x, P, N) = \sum_{e^+ \in P} -\log \frac{\exp\big(\text{sim}(x, e^+)\big)}{\sum_{e' \in N \cup \{e^+\}} \exp\big(\text{sim}(x, e')\big)}, \tag{2}$$

where $x$ is the query, $P$ is the positive set, and $N$ is the negative set. Additional hyperparameter details are provided in Appendix B.2.

At inference time (Figure 2-4), for each query, we first retrieve $k$ candidate examples from both our retriever and BM25 and merge the two lists. We then re-rank the merged candidates with the hybrid score introduced in §3.1, defined as the product of the two retrievers scores, and finally select the top $k$ examples as in-context examples. The number of retrieved examples does not have to be equal to the combination size used in CombiSearch.

### 3.3 Prompting with Text-to-JSON

This section describes our text-to-JSON prompt for DST, which we developed to avoid formatting errors observed when treating DST as a code generation task inspired by Liu et al. (2024b).

The prompt begins with a system message that instructs the model to act as a DST expert (Kong et al., 2024) and provides an explicit task definition (He et al., 2024). Next, domain-specific detailssuch as slots, types, and possible values for categorical slotsare provided. In-context examples follow, presented as tuples $\big([y^{t-1}, A^t, U^t], \Delta y^t\big)$ in which both the prior state $y^{t-1}$ and the state changes $\Delta y^t$ are represented in JSON format. The prompt ends with clear DST instructions and output format specifications, all formatted in Markdown for clarity (see Appendix G).

## 4 Experiments

In this section, we conduct ICL experiments for DST in various settings. The practical setting (§4.1) uses a trained retriever and evaluates DST performance on in-domain data (§4.1.1), unseen data (§4.1.2), and a dataset that involves coreference resolution (§4.1.3). On the other hand, the oracle setting (§4.2) examines the upper bound DST performance without retrieval errors. In ablation studies (§4.3), we assess the contribution of individual components in CombiSearch. Furthermore, we compare the combinatorial scoring method with an individual scoring baseline to measure the effect of modeling interactions among examples.

Common experimental settings are described below, with specific configurations detailed in their respective subsections. Further analysis of the experimental settings can be found in Appendix E.

**Common Experiment Settings** We evaluated our method on the MultiWOZ and Schema-Guided Dialogue (SGD) datasets. The MultiWOZ dataset comprises roughly 10,000 human-human dialogues across five domains (hotel, attraction, taxi, train, and restaurant) for training, with evaluation conducted on MultiWOZ 2.4. The SGD dataset contains 16,000 machine-machine dialogues across 26 services, with unseen domains in its test set used to assess generalization. Performance is measured using Joint Goal Accuracy (JGA), which requires exact matches between predicted and ground-truth dialogue state changes at each turn.

Our CombiSearch configuration uses a candidate pool size of $N = 100$ and $k = 10$ examples in each combination. We set $M = 3$ evaluations per example to balance accuracy with computational efficiency on large-scale datasets (Appendix E.2). During CombiSearch, we score candidate examples with `Llama-3-8B-Instruct` as the DST model. At inference time, we experiment with `Llama-3-8B-Instruct` for the DST model. To assess data efficiency, we explore using 1%, 5% and 100% of the training data, for both data construction for retriever training during CombiSearch and data retrieval at inference time. Further implementation details are in Appendix B.

### 4.1 Performance with Trained Retrievers

We evaluate CombiSearch's DST performance using retrievers fine-tuned on $D_{\text{CombiSearch}}$.

#### 4.1.1 In-domain Performance

We cover two scenarios: a **Closed-source** setting using black-box LLMs and an **Open-source** setting where only open-source LLMs are available.

| | Method | Language Model | MultiWOZ 2.1 1% | MultiWOZ 2.1 5% | MultiWOZ 2.1 100% | MultiWOZ 2.4 1% | MultiWOZ 2.4 5% | MultiWOZ 2.4 100% |
|---|---|---|---|---|---|---|---|---|
| Fine-tuned | DiSTRICT(Venkateswaran et al., 2023) | T5-small | 13.4 | 41.3 | 56.1 | - | - | - |
| | SGPDST(Lee et al., 2021) | T5-base | 32.1 | 43.1 | **56.7** | - | - | - |
| | SM2-11B(Chen et al., 2023) | T5-XXL | 38.4 | 44.6 | - | 40.0 | 51.1 | - |
| | DS2(Shin et al., 2022) | T5-large | 33.8 | 44.2 | 53.3 | 36.8 | 49.9 | 57.9 |
| ICL | IC-DST(Hu et al., 2022) | `code-davinci-002` | 43.1 | 47.1 | 48.7 | 50.7 | 48.4 | 55.4 |
| | SynthDST(Kulkarni et al., 2024) | `gpt-3.5-turbo` | **45.8** | 44.9 | 46.0 | 51.0 | 50.4 | 55.2 |
| | RefPyDST*(King and Flanigan, 2023) | `gpt-3.5-turbo` | 39.7 | 43.7 | 48.9 | 44.9 | 52.3 | 58.0 |
| | CombiSearch | `gpt-3.5-turbo` | **45.8** | **47.7** | 50.0 | **56.7** | **59.8** | **64.2** |

Table 1: JGA scores in the closed-source setting, evaluated on the full test sets of MultiWOZ 2.1 and 2.4 using 1%, 5%, and 100% of the training data. Each score represents the mean of three runs. *RefPyDST was reproduced under our own settings because `code-davinci-002` has been deprecated and the log-probabilities of input prompts are no longer provided by the API. More details are provided in Appendix B.1.

**Settings** The **Closed-source** setting is as follows. When we explore using only 1% and 5% of the training data, we run experiments three times with random samples of the training data (following Hu et al. (2022)). Unlike the common practice of training separate retrievers for each split (Hu et al., 2022; King and Flanigan, 2023), however, we train a single retriever on one split and use it for all three splits, which serve as retrieval corpora. This out-of-distribution (OOD) setup is more challenging than using a retriever trained specifically for each split, as the retriever must select effective examples from unseen data during inference. We use `gpt-3.5-turbo` as the DST model and report the mean performance across the three runs.

For **Open-source** experiments, we sample 1% and 5% of the training data once, which serve as both the retriever training data and the retrieval corpus. We experiment with `Llama-3-8B-Instruct` and a 4-bit GPTQ-quantized `Llama-3-70B-Instruct` (to accommodate resource constraints) as the DST model. Details about the baseline models used in the experiments are provided in Appendix B.1.

**Results** For the **Closed-source** setting, Table 1 shows that CombiSearch outperforms all baselines in data-constrained settings (1% and 5%) on both MultiWOZ 2.1 and 2.4. Notably, CombiSearch with 5% of training data even surpasses every baseline trained on the full dataset on MultiWOZ 2.4. Furthermore, with only 1% of training data, CombiSearch still exceeds SynthDST, which is trained on its full synthetic data. These results highlight the effectiveness and data efficiency of CombiSearch.

| **Retrieval** | **DST Model** | **1%** | **5%** | **100%** |
|---|---|---|---|---|
| RefPyDST | `Llama-3-8B-Instruct` | 47.7 | 50.6 | 55.5 |
| CombiSearch | `Llama-3-8B-Instruct` | **52.1** | **56.2** | **61.8** |
| RefPyDST | `Llama-3-70B-Instruct` | 56.0 | 57.1 | 56.2 |
| CombiSearch | `Llama-3-70B-Instruct` | **56.6** | **59.3** | **61.5** |
| RefPyDST | `gpt-3.5-turbo` | 46.2 | 52.7 | 58.0 |
| CombiSearch | `gpt-3.5-turbo` | 56.6 | 59.3 | 64.2 |

Table 2: JGA scores in the open-source setting on MultiWOZ 2.4, using 1%, 5%, and 100% of the training data. `gpt-3.5-turbo` performance on the same training data is also provided.

In the **Open-source** setting (Table 2), CombiSearch exceeds RefPyDST by more than 5% in JGA when `Llama-3-8B-Instruct` serves as the DST model, regardless of the amount of training data. A consistent—though smaller—average gain about 3% appears with the 4-bitquantized `Llama-3-70B-Instruct`. The last two rows of the table show the performance of `gpt-3.5-turbo` when the same training samples are used for a fair comparison, which exhibit the consistent tendency.

Although constructing CombiSearch data for training a retriever takes more wall-clock time than RefPyDST for the same training data (Appendix C), CombiSearch achieves equal or higher JGA in less overall time because it requires scoring far fewer training data. With only 5% of the training data, CombiSearch reaches 59.3% JGA, already surpassing RefPyDST trained on the full training data (56.2% JGA). Moreover, the running time of CombiSearch increases only linearly with the

| Training Dataset | Metric | Retrieval Method | | |
|---|---|---|---|---|
| | | Random | RefPyDST | CombiSearch |
| SGD | $y_{\text{test}}$ | 4.45 | 15.81 | 16.81 |
| | $\Delta y_{\text{test}}$ | 37.25 | 53.17 | 54.41 |
| MultiWOZ | $y_{\text{test}}$ | 4.45 | 12.08 | 12.24 |
| | $\Delta y_{\text{test}}$ | 37.25 | 49.16 | 50.16 |

Table 3: JGA scores on the SGD test set. For SGD and MultiWOZ rows, retrievers were trained on 5% of SGD and MultiWOZ 2.1, respectively.

number of examples to score, and this is only a one-time cost for retriever training that does not affect inference time. Therefore, in practical deployment settings where a retriever is updated only occasionally, the strong data efficiency and performance gains from CombiSearch compensate for its data construction cost.

#### 4.1.2 Generalization Ability

To verify that CombiSearchs advantage generalizes beyond MultiWOZ, we additionally evaluate on SGD dataset. We consider two experimental settings: (1) *in-domain*, where a retriever is trained and tested on SGD, and (2) *cross-dataset*, where a retriever trained on MultiWOZ is applied to SGD without adaptation.

**Settings** We train two retrievers on 5% of the training data: one on SGD and the other on MultiWOZ 2.1. Both were evaluated on 30% of the SGD test set. During inference, in-context examples were drawn from the same 5% subset of SGD, for a fair comparison. DST performance was measured with JGA for both the cumulative state ($y_{\text{test}}$) and the per-turn state change ($\Delta y_{\text{test}}$). The **Random** baseline selects examples randomly. `Llama-3-8B-Instruct` serves as the DST model.

**Results** Table 3 shows that both CombiSearch and RefPyDST generalize beyond their training corpus, outperforming the Random baseline by large margins. When the retriever is trained *in-domain (SGD)*, CombiSearch attains 54.41% JGA on $\Delta y_{\text{test}}$, edging out RefPyDST by roughly one 1% JGA and achieving a similar improvement on $y_{\text{test}}$ (16.81%). The same pattern holds under the *cross-dataset* setting: CombiSearch trained on MultiWOZ still surpasses its RefPyDST counterpart on both metrics. Though modest, this consistent edge under both domain-matched and domain-shift conditions underscores that CombiSearch is not merely overfitting to a single corpus. Rather, it preserves a performance lead across different domains.

#### 4.1.3 Coreference Resolution

We evaluate the effectiveness of CombiSearch in dialogues that require coreference resolution, a critical challenge in multi-turn, cross-domain DST. Because speakers routinely employ ellipsis or pronominalization, a system must resolve such contextual references and extract precise slot values from ongoing dialogues.

**Settings** Experiments are conducted on MultiWOZ 2.3, which provides turn-level coreference annotations (Han et al., 2021). Following King and Flanigan (2023), we compute (i) slot-level accuracy on turns that require coreference resolution and (ii) JGA restricted to dialogues that contain at least one such turn. For few-shot evaluation, we use 5% of the training data both to train the retriever and as the retrieval corpus. Two retrievers—RefPyDST and CombiSearch—are compared.

In this experiment, our focus is primarily on examining the differences in retrieved examples between the two methods. Therefore, we control for several factors that differ between them by using only embedding-based retrieval (instead of hybrid retrieval) and excluding the diversity-promoting re-ranking employed by RefPyDST. Since they use different prompt formats—Text-to-JSON ('JSON') and Text-to-Python ('Python')—we further control for prompt format. In the zero-shot condition (0%), we follow Hu et al. (2022) and provide a single example to inform the output format. `Llama-3-8B-Instruct` serves as the DST model.

**Results** Table 4 shows that, in the 5% setting, CombiSearch surpasses RefPyDST on accuracy and JGA by up to 2.7% (JSON 83.6% vs 81.6%; Python 23.6% vs 21.3%). This suggests that CombiSearch retrieves examples that are more helpful for coreference resolution than RefPyDST. In the zero-shot setting, the 'JSON' format yields better performance than the 'Python' format. This improvement may be due to frequent formatting errors in

the 'Python' prompt, as well as the superiority of our formatting instructions.

When a query requires coreference resolution, CombiSearch retrieved such examples slightly more often than RefPyDST (3.7 vs. 3.6 per query) and achieved about 2% higher JGA than RefPyDST (72.4% vs. 70.7%). These findings further highlight the strength of CombiSearch in retrieving effective examples for handling linguistic phenomena such as coreference resolution. Representative cases are provided in Appendix F.

| Prompt | Retriever | 5% | | 0% | |
|---|---|---|---|---|---|
| | | Acc | JGA | Acc | JGA |
| JSON | RefPyDST | 81.6 | 39.9 | 48.7 | 12.2 |
| | CombiSearch | **83.6** | **42.6** | | |
| Python | RefPyDST | 21.3 | 36.0 | 15.2 | 19.5 |
| | CombiSearch | **23.6** | **39.4** | | |

Table 4: Performance on coreference resolution data. **Acc**: the accuracy of slots for which coreference resolution is required. **JGA**: JGA for dialogue turns that require coreference resolution.

### 4.2 Upper Bound without Retrieval Errors

In this section, we run experiments to establish the *oracle* upper bound performance of CombiSearch by removing retrieval errors. Specifically, rather than using retrievers at inference time, we directly select the highest-scoring examples according to CombiScore with access to the query's ground-truth dialogue state. Although this setting is impractical in real-world applications, where the ground-truth dialogue states of queries are unavailable, it serves two key purposes: (i) estimating the maximal DST performance achievable from the retriever training data and (ii) assessing how effectively existing DST methods leverage available data.

Additionally, we conduct an error analysis using DST-EA, a tool we developed to categorize errors into seven types and twenty subclasses. This enables a deeper understanding of DST model behavior and contributes to further advancements in DST research. A detailed description of DST-EA and the error types is provided in Appendix A.1.

**Settings** Due to computational constraints, we use three separate 10% subsets of the test set and report the mean performance across these subsets to ensure statistical reliability. For each test query, we build $D_{\text{CombiSearch}}$ from the training set, exploring the use of either 1% or 5% of the data to examine the impact of example diversity on DST performance. We compare our method against two baselines: RefPyDST and a Hybrid approach that employs the pool construction process in CombiSearch and ranks examples based on the hybrid score (from BM25 and SBERT) without applying combinatorial scoring. For fair comparison, RefPyDST and Hybrid also skip retrieval errors and are given access to the ground-truth dialogue states of test queries. We employ both `Llama-3-8B-Instruct` and the 4-bitquantized `Llama-3-70B-Instruct` as the DST model.

**Results** As shown in Table 5, CombiSearch achieves 82.7% JGA with `Llama-3-8B-Instruct` (row 3) and 79.6% with the 4-bitquantized `Llama-3-70B-Instruct` (row 6). These scores represent an improvement of roughly 13% points over RefPyDST for each model (rows 1 and 4). The higher upper bound signifies greater headroom for the practical setting of using trained retrievers. It also indicates that the existing methods relied on suboptimal data for retriever training. Furthermore, CombiSearch outperforms the Hybrid baseline by 8%–12% (rows 2 and 5), highlighting the importance of aligning the scores of examples with DST performance through a combinatorial manner.

**Error Analysis** CombiSearch markedly reduces the most common errors made by BM25- and SBERT-based systems—chiefly, missing explicit user confirmations and overinterpreting ambiguous user utterances—by modeling context more comprehensively. Notable improvements were observed in recognizing confirmation utterances across multiple turns and achieving more accurate utterance interpretation.

### 4.3 Ablation Study

We evaluated the contributions of pool construction and combinatorial search through component ablation. In this analysis, we not only assess the significance of each component but explore the best way to integrate these two steps to maximize DST performance (see

| Scoring Method | Model Size | 1% | 5% | 100% |
|---|---|---|---|---|
| RefPyDST | 8B | 58.0 | 62.7 | 69.7 |
| Hybrid | 8B | 60.4 | 68.0 | 75.9 |
| **CombiSearch** | 8B | **68.4** | **75.1** | **82.7** |
| RefPyDST | 70B | 63.8 | 66.4 | 67.9 |
| Hybrid | 70B | 60.2 | 62.3 | 67.6 |
| **CombiSearch** | 70B | **71.8** | **72.7** | **79.6** |

Table 5: Oracle JGA scores across different scoring methods and models.

| Pool Construction | Combinatorial Search | # of Inference | JGA (%) |
|---|---|---|---|
| Random | CombiSearch | 31 | 56.9 |
| SBERT | CombiSearch | 31 | 67.7 |
| BM25&SBERT | CombiSearch | 31 | **70.7** |
| Random | N/A | 1 | 19.2 |
| SBERT | N/A | 1 | 60.8 |
| BM25&SBERT | N/A | 1 | 58.4 |
| - | $Se^2$ | 151 | 58.8 |

Table 6: Oracle JGA scores across different pool-construction and combinatorial search methods.

Appendix E for further analysis).

**Settings** All the experiments use the oracle setting as in §4.2, accessing test queries ground-truth dialogue states and directly selecting the highest-scoring examples. We use 5% of the training data and `Llama-3-8B-Instruct` as the DST model.

**Pool Construction** The original pool construction step of CombiSearch employs both BM25 and SBERT to retrieve example candidates. To analyze the contribution of each of these retrievers, we explore several configurations and measure DST performance.

As shown in Table 6, using both BM25 and SBERT (row 3) yielded a 14% higher JGA score than composing the pool with random examples (row 1). Additionally, using SBERT alone (row 2) led to a performance drop of 3%-4% compared to the combined approach. This finding underscores the importance of example diversity within the pool. Employing both retrievers results in examples that capture a wider range of dialogue characteristics and enables the combinatorial scoring step to effectively consider and recognize useful examples.

**Combinatorial Search** We compare CombiSearch with the another combinatorial search method $Se^2$ (Liu et al., 2024a), which greedily selects the example that maximally enhances ICL performance when combined with previously selected examples. For a fair comparison, we use the same three-shot setting as $Se^2$ ($k = 3$). While $Se^2$ employs a pool size of 50 examples, we use a smaller pool size of 30 ($N = 30$) for CombiSearch, which serves as a handicap. See Appendix B.1 for details about $Se^2$.

As shown in Table 6, CombiSearch outperformed $Se^2$ by 12% (rows 3 and 4). At the same time, CombiSearch is also more cost-efficient, requiring only 30 evaluations to construct $D_{\text{CombiSearch}}$ for a given query (i.e., 30 examples divided into 10 groups and evaluated 3 times), compared to the 150 evaluations needed by $Se^2$ to select three examples. The results demonstrate that while each component provides distinct benefits, their optimal integration substantially enhances the overall effectiveness of CombiSearch.

**Error Analysis** We observed a notable behavior for the DST model that was not present in the 10-shot ICL setting. The model initially predicted all domain-related slots as *dontcare* in the first turn before updating them in later turns. This suggests that, with fewer examples, the model overinterprets unspecified slot values as a lack of user preference.

### 4.4 Individual vs. Combinatorial Scoring

We test whether scoring examples in groups (CombiSearch) yields higher accuracy than scoring each example independently ($k = 1$ in CombiSearch).

**Setting** In the oracle setting, for every query, we draw 100 candidate dialogues from the full MultiWOZ training set (using ground-truth dialogue states) and score them with (i) *Individual* (100 calls/query), (ii) *CombiSearch* ($M$=3, 30 calls), and (iii) *CombiSearch* ($M$=9, 90 calls). In the practical setting, we use 5% of the training data under the open-source regime described in §4.1. Example scoring employs `Llama-3-8B-Instruct` as the DST model, and both `Llama-3-8B-Instruct` and 4-bitquantized `Llama-3-70B-Instruct` are tested during inference.

**Results** In the oracle setting (Table 7) CombiSearch with $M$=3 improves JGA by 2.8% over Individual while cutting compute to one-

| Scoring Method | Model | JGA (%) | Calls/query |
|---|---|---|---|
| Individual (100 cand.) | 8 B | 79.9 | 100 |
| CombiSearch ($M$=3) | 8 B | 82.7 | 30 |
| CombiSearch ($M$=9) | 8 B | 85.3 | 90 |

Table 7: Oracle JGA scores on the MultiWOZ test set across varying scoring methods.

| Scoring Method | Model Size | JGA (%) |
|---|---|---|
| Individual scoring | 8 B | 54.3 |
| CombiSearch ($M$=3) | 8 B | 56.2 |
| Individual scoring | 70 B | 56.9 |
| CombiSearch ($M$=3) | 70 B | 59.3 |

Table 8: Oracle JGA scores by individual and combinatorial scoring (5% training data).

third. With $M$=9, the gain widens to 5.4% at a comparable budget. The practical setting mirrors this pattern (Table 8): retrievers trained on CombiSearch data surpass their counterparts by roughly 3%, irrespective of the DST model's size. Hence, exploiting interactions among examples not only reduces computational cost but also results in higher-quality training data for retrievers.

## 5 Related Work

**Dialogue State Tracking** To ensure efficient domain transfer in DST, zero-shot methods have been explored (Wu et al., 2019; Hosseini-Asl et al., 2020; Gupta et al., 2022). But since they struggle when domains differ significantly, few-shot DST has been studied, where models are often fine-tuned to follow in-context demonstrations (Shin et al., 2022; Venkateswaran et al., 2023), with additional approaches exploring instruction tuning (Feng et al., 2023) and meta-training (Chen et al., 2023). However, these approaches require extensive training and annotation as the DST schema changes.

Another approach is to train retrievers to select in-context examples per query. Hu et al. (2022) labeled positive and negative examples and fine-tuned a retriever with the data. They proposed representing a dialogue history as a summary state and tracking state changes at each turn. Expanding on this, King and Flanigan (2023) applied maximum marginal relevance to diversify retrieved examples, while Kulkarni et al. (2024) proposed synthetic data generation for fine-tuning retrievers.

**In-Context Example Selection** Traditional methods often examples based on their impact on task performance (Nguyen and Wong, 2023; Chang and Jia, 2023). However, many studies use a fixed set of examples across different queries, limiting query relevance. To address this, other works train retrievers to select examples per query (Hu et al., 2022; King and Flanigan, 2023; Rubin et al., 2022). Our work presents a data construction method for training retrievers in this setting.

For data construction, studies have explored scoring functions to distinguish positive and negative examples. Common methods include measuring semantic similarity to the query, applicable to DST via dialogue state similarity (Liu et al., 2022; Hu et al., 2022; King and Flanigan, 2023). Other methods leverage the model's generation probabilities (Rubin et al., 2022; Iter et al., 2023; Peng et al., 2024). These generation probability-based methods suffer from surface-form competition (Holtzman et al., 2021). To mitigate this, while accounting for the model's impact, we optimize for end-task performance instead. Furthermore, some studies employ combinatorial search as in our work (Liu et al., 2023; Gupta et al., 2023). For instance, Liu et al. (2024a) suggest iteratively selecting the best example at a time in a greedy manner. We demonstrated that our method achieves a lower computational cost and higher effectiveness.

## 6 Conclusion

We presented CombiSearch, a combinatorial method for scoring in-context examples in DST. We explored the oracle upper bound of performance and demonstrated that CombiSearch-scored examples achieve a substantially higher JGA score than the SOTA model. In practical settings, the retriever trained on CombiSearch-scored examples also outperformed the SOTA model. Our error analysis tool sheds light on common error types made by LLMs. Our work highlights the importance of utilizing high-potential data for retriever training and identifies several areas for improvement in LLMs to reduce common errors.

## 7 Limitations

Our method facilitates effective example selection and training data preparation for DST re-

trievers. It also demonstrates high ideal performance, defining the upper bound for retriever training. This highlights its potential and suggests that existing methods rely on suboptimal data preparation in DST.

Applying in-context example selection to DST inevitably introduces computational overheada well-known challenge in such methods. Since existing approaches require evaluating thousands of candidates against a small example set, computational cost remains a major concern. To address this, we optimized efficiency while ensuring selection aligned with task objectives. As a result, we achieved strong discriminative power with only 30 evaluations on a large example set. Despite these optimizations, high computational overhead remains a key challenge for the field.

Additionally, practical retriever training did not fully reach the estimated upper bound. Our research primarily focuses on example selection and data preparation rather than optimizing retriever training itself. While designing advanced training methodologies is beyond our scope, recent studies have explored reinforcement learning-based iterative retrieval and contrastive learning for ranking. Future work could integrate such approaches to improve retriever performance. Advancements in this direction would not only enhance DST but also refine industrial applications.

## 8 Ethical and Social Impacts

**Social Implications** Our approach enhances dialogue state tracking efficiency and accuracy by optimizing the retrieval process via CombiSearch. This reduction in computational cost renders high-performing DST systems more accessible, particularly in resource-constrained settings, and promises to improve user experiences in domains such as customer service, healthcare, and education.

**Potential Risks and Challenges** Despite these benefits, risks remain. If the retrieval process selects irrelevant or low-quality examples, the DST system may yield misleading outputs, potentially propagating misinformation. Moreover, while our method is more efficient than traditional combinatorial approaches, the need for multiple evaluations still poses scalability challengesissues common to systems leveraging LLMs and example selection methods.

**Mitigation Strategies** To address these challenges, future work should focus on enhancing retriever robustness to consistently select high-quality, diverse examples, thereby reducing misinformation and improving domain generalization. Additionally, developing techniques to adapt to unseen or out-of-domain dialogue scenariossuch as augmenting training data or designing retrievers to prioritize variabilitywill be essential. Investigating adaptive evaluation strategies that minimize computational overhead without compromising performance is also a key direction.

By systematically addressing these concerns, our method aims to maximize positive impact while mitigating inherent risks, thus advancing the field of DST.

## Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) grant (RS-2024-00414981), by the Institute of Information & Communications Technology Planning & Evaluation (IITP) under the Leading Generative AI Human Resources Development grant (IITP-2025-RS-2024-00397085), and by the IITP grant (RS-2025-02215122, Development and Demonstration of Lightweight AI Model for Smart Homes), all of which were funded by the Korean government (MSIT). This research was also supported by the Korea Institute of Science and Technology Information (KISTI) in 2025 (No.(KISTI) K25L1M1C1), aimed at developing KONI (KISTI Open Neural Intelligence), a large language model specialized in science and technology.

## References

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Ad-*

*vances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc.

Ting-Yun Chang and Robin Jia. 2023. Data curation alone can stabilize in-context learning. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8123–8144, Toronto, Canada. Association for Computational Linguistics.

Derek Chen, Kun Qian, and Zhou Yu. 2023. Stabilized in-context learning with pre-trained language models for few shot dialogue state tracking. In *Findings of the Association for Computational Linguistics: EACL 2023*, pages 1551–1564, Dubrovnik, Croatia. Association for Computational Linguistics.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113.

Gordon V. Cormack, Charles L A Clarke, and Stefan Buettcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '09, page 758759, New York, NY, USA. Association for Computing Machinery.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Yujie Feng, Zexin Lu, Bo Liu, Liming Zhan, and Xiao-Ming Wu. 2023. Towards LLM-driven dialogue state tracking. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 739–755, Singapore. Association for Computational Linguistics.

Raghav Gupta, Harrison Lee, Jeffrey Zhao, Yuan Cao, Abhinav Rastogi, and Yonghui Wu. 2022. Show, don't tell: Demonstrations outperform descriptions for schema-guided task-oriented dialogue. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4541–4549, Seattle, United States. Association for Computational Linguistics.

Shivanshu Gupta, Matt Gardner, and Sameer Singh. 2023. Coverage-based example selection for in-context learning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 13924–13950, Singapore. Association for Computational Linguistics.

Ting Han, Ximing Liu, Ryuichi Takanobu, Yixin Lian, Chongxuan Huang, Dazhen Wan, Wei Peng, and Minlie Huang. 2021. Multiwoz 2.3: A multi-domain task-oriented dialogue dataset enhanced with annotation corrections and co-reference annotation. *Preprint*, arXiv:2010.05594.

Jia He, Mukund Rungta, David Koleczek, Arshdeep Sekhon, Franklin X Wang, and Sadid Hasan. 2024. Does prompt formatting have any impact on llm performance? *Preprint*, arXiv:2411.10541.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack William Rae, and Laurent Sifre. 2022. An empirical analysis of compute-optimal large language model training. In *Advances in Neural Information Processing Systems*.

Ari Holtzman, Peter West, Vered Shwartz, Yejin Choi, and Luke Zettlemoyer. 2021. Surface form competition: Why the highest probability answer isn't always right. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7038–7051, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Ehsan Hosseini-Asl, Bryan McCann, Chien-Sheng Wu, Semih Yavuz, and Richard Socher. 2020. A simple language model for task-oriented dialogue. In *Proceedings of the 34th International*

*Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA. Curran Associates Inc.

Yushi Hu, Chia-Hsuan Lee, Tianbao Xie, Tao Yu, Noah A. Smith, and Mari Ostendorf. 2022. In-context learning for few-shot dialogue state tracking. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 2627–2643, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Dan Iter, Reid Pryzant, Ruochen Xu, Shuohang Wang, Yang Liu, Yichong Xu, and Chenguang Zhu. 2023. In-context demonstration selection with cross entropy difference. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 1150–1162, Singapore. Association for Computational Linguistics.

Brendan King and Jeffrey Flanigan. 2023. Diverse retrieval-augmented in-context learning for dialogue state tracking. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 5570–5585, Toronto, Canada. Association for Computational Linguistics.

Aobo Kong, Shiwan Zhao, Hao Chen, Qicheng Li, Yong Qin, Ruiqi Sun, Xin Zhou, Enzhi Wang, and Xiaohang Dong. 2024. Better zero-shot reasoning with role-play prompting. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 4099–4113, Mexico City, Mexico. Association for Computational Linguistics.

Atharva Kulkarni, Bo-Hsiang Tseng, Joel Ruben Antony Moniz, Dhivya Piraviperumal, Hong Yu, and Shruti Bhargava. 2024. SynthDST: Synthetic data is all you need for few-shot dialog state tracking. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1988–2001, St. Julian's, Malta. Association for Computational Linguistics.

Chia-Hsuan Lee, Hao Cheng, and Mari Ostendorf. 2021. Dialogue state tracking with a language model using schema-driven prompting. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 4937–4949, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Itay Levy, Ben Bogin, and Jonathan Berant. 2023. Diverse demonstrations improve in-context compositional generalization. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1401–1422, Toronto, Canada. Association for Computational Linguistics.

Haoyu Liu, Jianfeng Liu, Shaohan Huang, Yuefeng Zhan, Hao Sun, Weiwei Deng, Furu Wei, and Qi Zhang. 2024a. $se^2$: Sequential example selection for in-context learning. In *Findings of the Association for Computational Linguistics ACL 2024*, pages 5262–5284, Bangkok, Thailand and virtual meeting. Association for Computational Linguistics.

Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. 2022. What makes good in-context examples for GPT-3? In *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures*, pages 100–114, Dublin, Ireland and Online. Association for Computational Linguistics.

Jinxin Liu, Li He, Yachen Kang, Zifeng Zhuang, Donglin Wang, and Huazhe Xu. 2023. Ceil: generalized contextual imitation learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, Red Hook, NY, USA. Curran Associates Inc.

Michael Xieyang Liu, Frederick Liu, Alexander J. Fiannaca, Terry Koo, Lucas Dixon, Michael Terry, and Carrie J. Cai. 2024b. "we need structured output": Towards user-centered constraints on large language model output. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, CHI EA '24, New York, NY, USA. Association for Computing Machinery.

Pan Lu, Liang Qiu, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, Tanmay Rajpurohit, Peter Clark, and Ashwin Kalyan. 2023. Dynamic prompt learning via policy gradient for semi-structured mathematical reasoning. In *ICLR*.

Tai Nguyen and Eric Wong. 2023. In-context example selection with influences. *Preprint*, arXiv:2302.11042.

Keqin Peng, Liang Ding, Yancheng Yuan, Xuebo Liu, Min Zhang, Yuanxin Ouyang, and Dacheng Tao. 2024. Revisiting demonstration selection strategies in in-context learning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9090–9101, Bangkok, Thailand. Association for Computational Linguistics.

Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China. Association for Computational Linguistics.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: Bm25 and beyond. *Found. Trends Inf. Retr.*, 3(4):333389.

Ohad Rubin, Jonathan Herzig, and Jonathan Berant. 2022. Learning to retrieve prompts for in-context learning. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2655–2671, Seattle, United States. Association for Computational Linguistics.

Jamin Shin, Hangyeol Yu, Hyeongdon Moon, Andrea Madotto, and Juneyoung Park. 2022. Dialogue summaries as dialogue states (DS2), template-guided summarization for few-shot dialogue state tracking. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 3824–3846, Dublin, Ireland. Association for Computational Linguistics.

Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. 2020. Mpnet: Masked and permuted pre-training for language understanding. *Preprint*, arXiv:2004.09297.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open foundation and fine-tuned chat models. *Preprint*, arXiv:2307.09288.

Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2019. Representation learning with contrastive predictive coding. *Preprint*, arXiv:1807.03748.

Praveen Venkateswaran, Evelyn Duesterwald, and Vatche Isahagian. 2023. DiSTRICT: Dialogue state tracking with retriever driven in-context tuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5101–5112, Singapore. Association for Computational Linguistics.

Chien-Sheng Wu, Andrea Madotto, Ehsan Hosseini-Asl, Caiming Xiong, Richard Socher, and Pascale Fung. 2019. Transferable multi-domain state generator for task-oriented dialogue systems. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 808–819, Florence, Italy. Association for Computational Linguistics.

Jiacheng Ye, Zhiyong Wu, Jiangtao Feng, Tao Yu, and Lingpeng Kong. 2023. Compositional exemplars for in-context learning. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*.

Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2).

## A Error Analysis Framework

We introduce a comprehensive error analysis tool that categorizes DST prediction errors into seven types, allowing us to better understand the underlying causes of mistakes made by LLMs and inform directions for improving the methods.

Figure 3 shows our classification of error types (error types are marked in boxes). Recall that the predicted dialogue state in the current turn $t$ is $y^t = \Delta y^t \cup y^{t-1}$. Hence, at the top level, we classify errors based on two sources: (1) misprediction of slot-value pairs from the utterances in the current turn $\Delta y^t$ (*delta*), or (2) propagation of mispredictions from past turns $y^{t-1}$ (*error propagation*). These two sources share the same set of specific error types.

At the second level, we classify errors by comparing slot-value pairs between the ground-truth dialogue state and the prediction: (1) a slot-value pair in the ground-truth is absent in the prediction (*miss*), and (2) a slot-value pair in the prediction is absent in the ground-truth (*hallucination*). These two categories contain seven specific error types, detailed as follows.

For the *miss* category, an error may arise because the prediction (1) misses an entire slot-value pair (*miss-total*), (2) misses a special token "delete" (*miss-delete*), (3) misses "don't care," which indicates the user's openness to any options (*miss-dontcare*), or (4) misses a slot by confusing it with another slot (*miss-confuse*).

In the *hallucination* category, errors may arise when the prediction (1) hallucinates a slot-value pair that is not informed by the dialogue (*hallucination-total*), (2) hallucinates a value with a correct slot that does not match the dialogue (*hallucination-value*), or (3) incorrectly modifies the previous dialogue state (*hallucination-overwrite*).

This categorization systematically identifies various sources of errors and provides an intuitive framework for understanding DST model weaknesses. Additionally, it highlights the challenges faced by LLMs in key DST capabilities, such as utterance comprehension, hallucination detection, and identifying critical slots. We release the framework as a Python library to facilitate its easy integration in future studies.

### A.1 Error Analysis of Modern LLMs

To understand the weaknesses of modern LLMs in DST, we used our error analysis framework to evaluate the zero-shot and 10-shot DST performance of four models: RefPyDST (King and Flanigan, 2023), `gpt-3.5-turbo`, `Llama-3-8B-Instruct`, and the 4-bit quantized `Llama-3-70B-Instruct` (Dubey et al., 2024). RefPyDST, the current SOTA model, was reproduced with `Llama-3-8B-Instruct` (Dubey et al., 2024) because `code-davinci-002` has been deprecated. It uses prompts in Python format, while the other models employ natural language prompts. The prompts used in the experiments are shown in Appendix G.

In the 10-shot setting, RefPyDST retrieves in-context examples using its own fine-tuned retriever. For the other models, we randomly selected examples from the full training dataset.

Overall, *error propagation* errors were significantly more frequent than *delta* errors. In the zero-shot setting, *error propagation hallucination-total* was the most common, while in the random setting, *error propagation miss-total* dominated. For RefPyDST, an exception was observed where format errors were the most frequent due to Python code formatting, with *hallucination-total* being the next most prominent type of error.

In the 10-shot setting, *miss* errors outnumbered *hallucination* errors in all models except `gpt-3.5-turbo`. In the zero-shot setting, hallucination errors were generally more prevalent. Across all models, *miss-total* and *hallucination-total* errorswhere the correct slot and value are either missed or incorrectly generatedwere the most frequent. *Miss-total* errors affected subsequent dialogue turns more extensively, especially for name-related slots, due to issues like user confirmation and flexible preferences not being properly interpreted. Meanwhile, *hallucination-total* errors often stemmed from oversensitivity to system descriptions, incorrect coreference resolution, or user question misinterpretation, notably affecting area-related slots.

To reduce both miss and hallucination errors, addressing *delta* errors and ensuring accurate understanding of current dialogue turns are essential. We now incorporate both the dialogue state and conversation similarity in retrieval and provide explicit instructions in the prompts for better information extraction.

## B Experiment Settings

### B.1 Baselines

**DiSTRICT (Venkateswaran et al., 2023)** Retrieves highly relevant examples based on semantic similarity within dialogues to enhance in-context tuning of T5-small for predicting slot values, eliminating the need for handcrafted templates.

**SGPDST (Lee et al., 2021)** Applies schema-driven prompting by providing both dialogue context and schema information into the encoder of a pre-trained T5-base. Its prompts consist of candidate slot labels and their natural language descriptions.

**SM2 (Chen et al., 2023)** Adapts a meta-ICL framework by training T5-XXL on support dialogue datasets in DST format and evaluating on MultiWOZ 2.4. It aims to stabilize few-shot performance and reduce variance

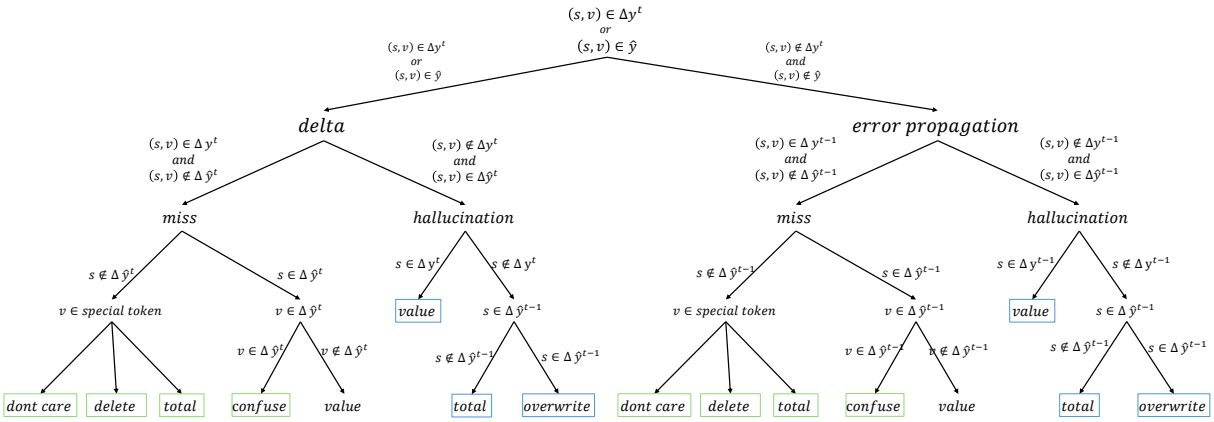



Figure 3: Error tree to divide error type by slot and value

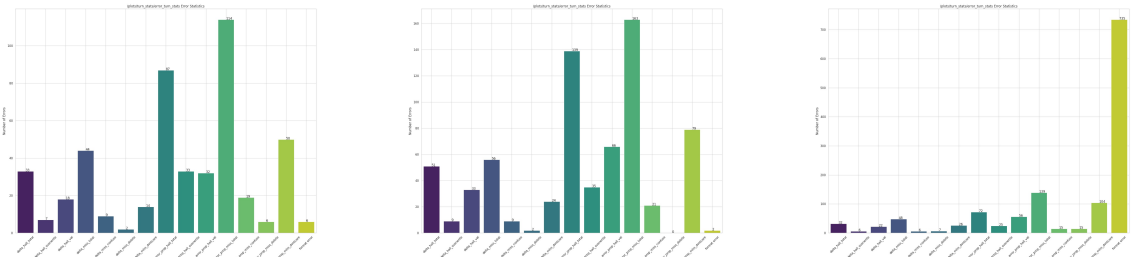


Figure 4: Statistics of error types made by different models in §4.2. From left to right: CombiSearch, Hybrid, and RefPyDST. Formatting errors in RefPyDST are not included.

across prompts. It fine-tunes SBERT as an example retriever and uses the predicted dialogue state as a summary of dialogue history.

**DS2 (Shin et al., 2022)** Reframes dialogue state tracking as a summarization task, fine-tuning T5-large and BART-large to generate rule-based synthetic summary templates.

**IC-DST (Hu et al., 2022)** Applies ICL for DST by reformulating the task as a text-to-SQL generation, using SQL to represent dialogue states. An example retriever is trained by scoring examples based on the similarity of their dialogue state changes, and fine-tuning SBERT. Additionally, prior dialogue states are used as context instead of the full dialogue history.

**SynthDST (Kulkarni et al., 2024)** Builds on Hu et al. (2022), which reformulated DST as a text-to-SQL task. Unlike Hu et al. (2022), who fine-tuned retrievers for semantically similar dialogues, SynthDST generates high-quality synthetic datasets using LLMs without retriever fine-tuning. With `code-davinci-002` deprecated, `gpt-3.5-turbo` is used to maintain system performance.

**RefPyDST (King and Flanigan, 2023)** Follows the ICL framework for DST introduced by Hu et al. (2022). It reformulates DST as a text-to-Python task and trains a retriever using the two-step data generation process from Hu et al. (2022). To enhance inference diversity, it employs maximum marginal relevance for example decoding and PMI-based decoding to mitigate surface form competition. With `code-davinci-002` deprecated, we replaced it with `gpt-3.5-turbo` and substituted $\text{PMI}^{\beta}$ decoding with beam search.

**$\text{Se}^2$ (Liu et al., 2024a)** Selects in-context examples sequentially through beam search rather than the traditional 'select and organize' approach. This method leverages LLM feedback to capture inter-example relationships and sequential patterns, improving the contextual relevance of ICL prompts. For DST, we applied this approach using Joint F1 as the evaluation metric and `Llama-3-8B-Instruct` as the LLM to provide task-specific feedback. To reduce computational overhead, we set the beam size to 1 for

beam search.

### B.2 Hyperparameters

All hyperparameter tuning was conducted on the 5% validation set and performed manually. The parameters used in the CombiSearch process are presented in the "Common Settings" paragraph of §4. During inference, we configured both BM25 and SBERT to retrieve the top 10 highest-scoring examples. For LLM inference, beam search was applied as the decoding strategy, with the beam width set to 4. For retriever training, the batch size was set to 32. For each instance, we set the number of hard negatives $B$ to 16, the number of negatives to 15, and the number of positives to 1. Consequently, each training instance consists of 1 positive, 16 hard negatives, and 15 negatives.

## C Computing Details

| Train Data Size | # of instance | Running Time (h) |
|---|---|---|
| 1% | 524 | 5 |
| 5% | 2731 | 26 |
| 100% | 54971 | 550 |

Table 9: The number of instances corresponding to 1%, 5%, and 100% training data and the time required to generate $D_{\text{CombiSearch}}$ by applying CombiSearch to these datasets.

| Training Method | 1% (hour) | 5% (hour) | 100% (hour) |
|---|---|---|---|
| CombiSearch | 4 | 20 | 260 |
| RefPyDST | 4.5 | 18 | 36.5 |

Table 10: Total training time for training retrievers based on each method. We report the total time required to train a retriever, so the running time of RefPyDST (King and Flanigan, 2023) for 1% and 5% is the aggregated training time of the three retrievers.

**Hardware** All experiments were run on a single NVIDIA A6000 for `Llama-3-8B-Instruct` and a single NVIDIA A100 for 4-bitquantized `Llama-3-70B-Instruct`. Inference speed was accelerated with vLLM; the 70 B model was additionally compressed with GPTQ (4-bit, group size 128) to fit on one GPU without noticeable accuracy loss.

**Inference cost** Table 9 lists the total wall-clock time required to construct $D_{\text{CombiSearch}}$ under the 1%, 5%, and 100% training regimes.

**Retriever training** Training is performed on a single NVIDIA A100. For each dialogue $x_i$ we create 10 positive instances and 31 negatives (ğ3.2). Because the large negative pool increases per-epoch cost, we train for 15 epochs on the 1% and 5% splits and for 7 epochs on the 100% split. Detailed statistics are given in Table 10.

**Comparison with RefPyDST.** For the same amount of training data, CombiSearch takes 4.516.2x as much time as RefPyDST. The computation time for CombiSearch **increases linearly with the training data size** as expected, because CombiSearch is not designed to explore all possible combinations. On the other hand, RefPyDST turns out to benefit from a speed boost as training progresses. RefPyDST trains *three* separate retrieversone per data shardwhereas CombiSearch trains a *single* model on the full split. This design choice makes CombiSearch slower to train (e.g. 260 h vs. 36.5 h at 100%), yet the method still reaches higher Joint Goal Accuracy with only 5% of the data. In practice, its stronger generalization shortens the overall *time-to-target-accuracy* despite the longer training phase. Crucially, however, the extra compute buys markedly better sample efficiency: with only 1% of the data, CombiSearch already exceeds RefPyDST trained on 5%, and with open-source inference models it even beats RefPyDST trained on the full corpus. Since pre-training is a one-off expense and retrieval latency at inference time is identical across methods, the longer training phase is quickly amortised; in realistic deployments, months-to-years of serving time dominate the few hundred hours needed to build the CombiSearch retriever, making it the faster route to any target Joint Goal Accuracy.

## D Retriever Details

In this section, we present the details related to our retriever.

First, in §D.1, we provide a detailed explanation of how each retriever's input is formatted during both training and inference. In Ap-

pendix D.2, we provide experimental justification for the approach in which BM25 and SBERT are assigned different input components for the same query, as described in §3.1.

### D.1 Embedding Model Input

There are three key stages where the embedding models input plays a crucial role: (1) when applying CombiSearch, (2) when training on data obtained from CombiSearch, and (3) when using the trained retriever for inference. We detail how the retrievers input is structured in each case.

During the CombiSearch process, all examples are embedded using only their state change information. Specifically, to represent an example $\{x_i, \Delta y_i\}$, we use only $\Delta y_i$ for embedding. This applies equally to both training instances and query instances, which are sampled from the training set.

We trained an embedding model on $D_{\text{CombiSearch}}$ to select examples that enhance DST performance when combined in the prompt (Figure 2-3) [2]. Each query in $D_{\text{CombiSearch}}$ is paired with a candidate pool of size $N$, where each example is a dialogue context and dialogue state change pair, $\big(e_i = (x_i, \Delta y_i)\big)$. Following prior work (Hu et al., 2022; King and Flanigan, 2023), we concatenate the dialogue context $y^{t-1}$ and recent utterances $A^t, U^t$ before encoding a turn context $x_i$ (or query) with the embedding model *emb*. To measure relevance between two input instances $x_i$ and $x_j$, we compute $\cos\big(emb(x_i), emb(x_j)\big)$.

At inference time, we similarly embed a test instance $x_t$ and perform a nearest-neighbor search to retrieve the most relevant examples for the test dialogue context.

### D.2 Optimizing Retriever Inputs for Hybrid Search in CombiSearch

We conducted experiments to determine the optimal input format for each retriever.

**Setting** Few-shot DST was evaluated on a 20% validation set using 5% of the training data, assuming oracle settings with access to the gold state. We assessed suitable inputs for BM25 and SBERT in hybrid search (Table 11, Table 12). Based on these results, we selected the two best input formats for each retriever and applied CombiSearch with combinatorial example scoring to evaluate overall performance (Table 13). Both retriever scores were z-score normalized and multiplied for reranking, following the procedure in §4.

[2]We use all-mpnet-base-v2 (Song et al., 2020)

| Index | History | Recent Utterances | Gold State | JGA (%) |
|---|---|---|---|---|
| i | O | O | $\Delta y_{slot}$ | 55.6 |
| ii | X | O | $\Delta y_{slot}$ | 55.7 |
| iii | X | X | $\Delta y_{slot}$ | **66.8** |
| iv | O | X | $\Delta y_{slot}$ | 50.9 |

Table 11: The JGA using BM25 varied its input. 'O' indicates inclusion, while 'X' indicates exclusion. $\Delta y_{slot}$ represents the slots of state change.

| Index | History | Recent Utterances | Gold State | JGA (%) |
|---|---|---|---|---|
| i | O | O | $\Delta y$ | 44.4 |
| ii | X | O | $\Delta y$ | 44.2 |
| iii | X | X | $\Delta y$ | **67.3** |
| iv | O | X | $\Delta y$ | 42.0 |

Table 12: The JGA using SBERT varied its input. 'O' indicates inclusion, while 'X' indicates exclusion. $\Delta y$ represents the state change.

**Result** As shown in Table 11, BM25 performed best when retrieving examples based only on the slot values of the query's gold state change (iii), followed by using only the utterance and state change slots without history (ii). Similarly, Table 12 shows that SBERT achieved the highest performance when retrieving examples solely based on the query's gold state change (iii), with the next best approach incorporating history, utterance, and state change (i).

To assess hybrid search, we combined the top two input formats from each retriever, resulting in four experimental conditions (Table 13). The best-performing combination used BM25 with utterance and state change slots (excluding history) (ii) and SBERT with examples retrieved based on the query's gold state change (iii). We adopted this combination for pool construction in CombiSearch.

| Input of BM25 | Input of SBERT | JGA (%) |
|---|---|---|
| ii | i | 57.7 |
| ii | iii | **64.8** |
| iii | i | 58.8 |
| iii | iii | 58.8 |

Table 13: The oracle JGA using SBERT and BM25 varied its input. We aggregated the two scores using multiplication. Ultimately, to determine which input to use in the CombiSearch data preparation process, we conducted experiments under the same settings. The Roman numeral (e.g., i) in the inputs of BM25 and SBERT refers to the index of input combinations presented in the previous tables. Refer to Table 11 for BM25 input and Table 12 for SBERT input.

| Evaluations per example | 1% (JGA) | 5% (JGA) | Running time (h) | Est. build time (h) |
|---|---|---|---|---|
| 1 | 65.02 | 69.67 | 2.3 | 180 |
| 2 | 68.98 | 72.81 | 4.2 | 330 |
| 3 | 69.12 | 75.27 | 6.0 | 470 |
| 5 | 69.94 | 78.14 | 10.0 | 769 |
| 10 | 76.91 | 80.87 | 19.5 | 1515 |

Table 14: Oracle-mode JGA and compute cost as a function of the evaluation budget $M$. "Running time" measures the 10% test slice; "Est. build time" extrapolates to the full training set.

## E Further Analysis

Here, we conduct a more in-depth analysis. First, we investigate the effect of the number of in-context examples in Appendix E.1. Next, we conduct an experiment by increasing the number of evaluations ($M$) for each example in the example scoring process, as described in Appendix E.2. Finally, we examine the score aggregation method that integrates the scores returned by BM25 and SBERT in Appendix E.3.

### E.1 The Effects of the Number of In-Context Examples

Here, we modify the number of in-context examples included in the prompt under both the oracle setting, where the query's gold state change is accessible for retrieval, and the practical setting.

**Setting** In both settings, we use 100% of the training data as the corpus and evaluate performance on a 20% validation set. We vary the number of in-context examples from 1 to 10 and measure the corresponding performance. We compare BM25, SBERT, and BM25&SBERT, where SBERT is used without fine-tuning. In the oracle setting, the slot values of the gold state change are used as input for BM25, while SBERT takes the gold state change as input. In the practical setting, both BM25 and SBERT take the dialogue context, represented as the dialogue state and the most recent utterance, as input.

**Result** The results in Figure 5 show that performance tends to improve as more shots are included, without noticeable saturation up to 10 shots. This trend is observed across BM25, SBERT, and BM25&SBERT. The performance gap between the oracle and practical settings was larger for SBERT than for BM25, suggesting that SBERT is more dependent on access to the gold state. Additionally, SBERT's influence appears to have significantly contributed to the high performance of BM25&SBERT in the oracle setting.

### E.2 Trade-off Between Accuracy and Cost of the Evaluation Budget $M$

We assess how the per-example evaluation budget $M$ influences oracle setting performance and compute cost, with the goal of identifying an $M$ that offers the best balance between JGA and wall-clock time.

**Setting** The experiment follows the CombiSearch configuration in §4 under oracle setting that has access to the gold dialogue state change (oracle mode). Few-shot DST is evaluated on 1% and 5% subsamples of the MultiWOZ training set using a 10% test slice. The estimated *build time* for the full training set is obtained by multiplying the number of training dialogues by the average latency of scoring each instance $M$ times.

**Results** Table 14 shows that accuracy rises linearly with $M$, while runtime scales almost linearly. At 5% data, increasing $M$ from 3 to 5 adds +2.87 absolute point JGA but inflates runtime by 66% and build cost by 64%. With only 1% data, the same change yields a negligible +0.82% absolute point. Hence, $M = 3$ **emerges as the most cost-effective point**, capturing the bulk of the attainable accuracy without incurring prohibitive compute.

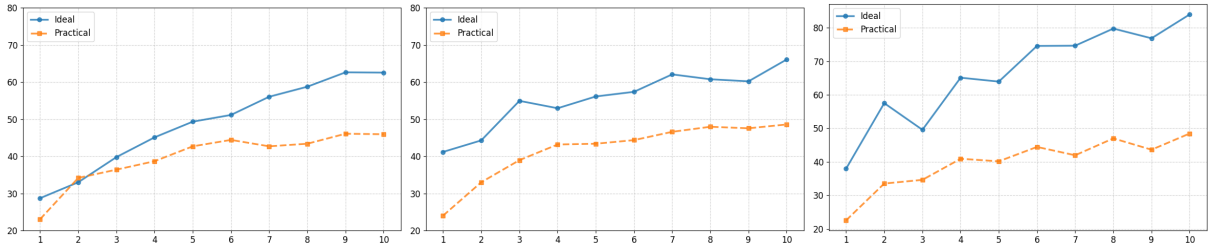



Figure 5: Changes in JGA according to the number of shots. From left to right: BM25, SBERT, and BM25&SBERT. The "ideal" setting refers to a scenario where access to the gold state is available, while the "practical" setting refers to a scenario where it is not. Interestingly, when using BM25&SBERT, the performance gap between the oracle and practical settings was the largest.

### E.3 Score Aggregation Methods

We compared the performance of two retrievers based on different score aggregation methods.

**Setting** Few-shot DST was conducted using 5% of the training data and evaluated on three test sets, each comprising 10% of the full test data. The final performance was determined by averaging the results across these sets. This experiment was conducted after all design choices were finalized to assess the effectiveness of our aggregation method. Since the focus was on evaluating score aggregation, we assumed access to the gold state, eliminating retrieval errors from consideration as in §4.2.

We re-ranked retrieved examples using SBERT and BM25 scores through the following methods:

**Multiplication** The approach selected prior to this experiment, Multiplication, combines SBERT and BM25 scores by multiplying them. Due to scale differences, direct multiplication is infeasible. Instead, both scores are first normalized using z-score transformation before multiplication.

**Weighted Sum** This method sums SBERT and BM25 scores after applying z-score transformation to account for scale differences. Equal weighting is used, setting $\alpha = 0.5$.

**RRF (Cormack et al., 2009)** Reciprocal Rank Fusion (RRF) assigns a score based on the reciprocal of an examples rank in each retrievers list. These scores are then summed across both retrievers. Unlike z-score normalization, RRF does not alter score distributions, making it a widely used and simple re-ranking approach.

| Aggregation Method | JGA (%) |
|---|---|
| **Multiplication** | **75.063** |
| Weighted Sum | 71.0 |
| RRF | 70.8 |
| Union Top K | 67.7 |

Table 15: The result of score aggregation method.

**Union Top-K** This method selects the top-K examples by alternating between SBERT and BM25s ranked lists in a round-robin fashion. If a duplicate appears earlier in the ranking, the next available example is selected instead.

**Result** Multiplication delivers markedly better performance than the alternative methods, confirming that our design choice was appropriate.

## F An Example of Retrieved Example

We showcase an example in Table 16. We observe substantial overlap between the querys previous dialogue state and that of the CombiSearch example. Both dialogues contain complex prior states involving two entities—restaurant and attraction in the query, restaurant and hotel in the example—before the user transitions to a new request (booking a taxi). Notably, the CombiSearch example mirrors the querys dialogue history and therefore presents a user utterance ("i also need a taxi..") that exhibits a similar structural shift: it refers back to two previously mentioned points of interest (departure and destination) using coreference. Resolving such coreferences is challenging in DST, and in-context examples that display this pattern can enhance model performance. By contrast, although the RefPyDST

| Example Type | Category | Example |
|---|---|---|
| **Query** | **Previous Dialogue State** | restaurant-book day: saturday, restaurant-book people: 7, restaurant-book time: 17:15,restaurant-food: spanish, restaurant-name: la raza, restaurant-area: centre, attraction-name: saint barnabas press gallery |
| | **System Utterance** | 'saint barnabas press gallery s postcode is cb13ew . would you like more information ?' |
| | **User Utterance** | 'no that s it . i also need a taxi taking me from the gallery to the restaurant .' |
| | **Dialogue State Change** | taxi-destination: la raza, taxi-departure: saint barnabas press gallery |
| **CombiSearch Example** | **Previous Dialogue State** | restaurant-book day: wednesday, restaurant-book people: 6, restaurant-book time: 16:15, restaurant-food: italian, restaurant-name: prezzo, restaurant-area: west, hotel-book day: wednesday, hotel-book people: 6, hotel-book stay: 1, hotel-name: huntingdon marriott hotel, hotel-area: west, hotel-pricerange: expensive |
| | **System Utterance** | "i was able to successfully book you for 1 night . your reference number is bwxli8k3 . is there anything more i can help you with ?' |
| | **User Utterance** | 'i also need a taxi to get from the restaurant to the hotel .' |
| | **Dialogue State Change** | taxi-destination: huntingdon marriott hotel, taxi-departure: prezzo |
| **RefPyDST Example** | **Previous Dialogue State** | restaurant-name: pipasha restaurant, attraction-name: club salsa, |
| | **System Utterance** | 'pipasha is in the east . the full address is newmarket road fen ditton postal code cb58pa . can i help you with anything else ?' |
| | **User Utterance** | 'i am gonna need a taxi to get from the club to the restaurant . can you schedule that for me ?' |
| | **Dialogue State Change** | taxi-destination: pipasha restaurant, taxi-departure: club salsa |

Table 16: Representative example retrieved by each retriever for the target query. The CombiSearch example shares substantial Previous Dialogue State with the query. This indicates that CombiSearch effectively retrieves examples that share not only dialogue state change but also a similar conversational flow and structural context.

example also contains a taxi request between two entities, its preceding dialogue state overlaps far less with the querys rich context and lacks the nuanced history captured by CombiSearch.

This case illustrates that our hybrid CombiSearch retriever finds examples sharing not only dialogue-state similarity but also a parallel conversational flow and structure, leading to significant performance gains.

## G Prompt for Few-shot DST experiments

| Role | Message |
|---|---|
| **System** | **Task:** You are an expert in Dialogue State Tracking (DST) focused on managing and updating the dialogue state change based on system-user interactions. The dialogue state represents the user's preferences and booking details across different domains: Hotel, Train, Attraction, Restaurant, and Taxi. |
| **User** | **Guidelines:**<br>1. **Hotel:**<br>- **Slots:** name (string), pricerange (PriceRange), type (HotelType), parking (Option), book stay (integer), book day (DayOfWeek), book people (integer), area (Area), stars (integer between 0 and 5 or "dontcare"), internet (Option).<br>- **Valid Values:**<br>- PriceRange: "dontcare", "cheap", "moderate", "expensive".<br>- HotelType: "hotel", "guest house", "dontcare".<br>- Option: "yes", "no", "dontcare".<br>- DayOfWeek: "monday", "tuesday", "wednesday", "thursday", "friday", "saturday", "sunday".<br>- Area: "dontcare", "centre", "east", "north", "south", "west".<br><br>2. **Train:**<br>- **Slots:** destination (string), departure (string), day (DayOfWeek), book people (integer), leaveat (hh:mm or "dontcare"), arriveby (hh:mm or "dontcare").<br><br>3. **Attraction:**<br>- **Slots:** name (string), area (Area), type (AttractionType).<br>- **Valid Values:**<br>- AttractionType: "architecture", "boat", "church", "cinema", "college", "concert hall", "entertainment", "hotspot", "multiple sports", "museum", "nightclub", "park", "special", "swimming pool", "theatre", "dontcare".<br><br>4. **Restaurant:**<br>- **Slots:** name (string), food (string), pricerange (PriceRange), area (Area), book time (hh:mm or "dontcare"), book day (DayOfWeek), book people (integer).<br><br>5. **Taxi:**<br>- **Slots:** destination (string), departure (string), leaveat (hh:mm or "dontcare"), arriveby (hh:mm or "dontcare").<br><br>**Example 1 of Dialogue State Change Update Task:**<br>**Previous Belief State:**<br>{'train-destination': 'cambridge', 'train-day': 'thursday', 'train-arriveby'. '15:45', 'train-departure': 'kings ly<br><br>**Latest Conversation Between System and User:**<br>**System:** "It leaves at 14:11 and lasts 47 minutes. Is there anything else?"<br>**User:** "Yes, I am looking for a restaurant that serves North American food, preferably in the expensive price range."<br><br>**Instructions:**<br>- Based on the user's latest input, update the belief state by correctly identifying and filling in the relevant domain(s), slot(s) and value(s).<br>- Provide your output strictly in the Required Output Format below.<br><br>**Required Output Format:**<br>**Dialogue state change after Latest Conversation Between System and User:**<br>{ "DOMAIN_1- SLOT_1": "VALUE_1", "DOMAIN_2-SLOT_2": "VALUE_2", ... } |
| **Assistant** | {'restaurant-food': 'north american', 'restaurant-pricerange': 'expensive'} |

Table 17: An example of prompt for few-shot DST. For brevity, this example only includes one example without query. However, we set ten shots as a default setting for fair comparison with baseline models.